



# Manual técnico

## *Aurora Viajes*

Plataforma web de reservas de viajes construida con React + Vite, FastAPI y MySQL, desplegada en Cloudflare y Railway, con pagos Stripe, correo transaccional e inteligencia artificial.

#### AUTOR

David Alejandro Barrientos Bedoya

#### INSTRUCTOR

Jhan Hader Muñoz

#### COMPETENCIA

React + Vite / FastAPI

#### VERSIÓN

1.0 · 24 de septiembre de 2026

#### INSTITUCIÓN

SENA · Regional Antioquia · Centro de Servicio y Gestión Empresarial

<https://aurora-viajes.carousel-app.workers.dev>  
<https://aurora-backend-production-a2b4.up.railway.app/api>  
<https://github.com/davidabb2000/aurora-viajes>

CONTROL DEL DOCUMENTO

| Versión | Fecha                    | Autor                             | Descripción                                                                                                              |
|---------|--------------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 24 de septiembre de 2026 | David Alejandro Barrientos Bedoya | Primera versión: arquitectura, modelo de datos, API, seguridad, módulos, configuración, pruebas, despliegue y operación. |

CONTENIDO

|     |                                           |    |
|-----|-------------------------------------------|----|
| 1   | Introducción                              | 3  |
| 1.1 | Propósito del documento                   | 3  |
| 1.2 | Alcance                                   | 3  |
| 1.3 | Público objetivo                          | 3  |
| 1.4 | Convenciones                              | 3  |
| 1.5 | Documentos relacionados                   | 3  |
| 2   | Descripción general del sistema           | 5  |
| 2.1 | Qué hace la plataforma                    | 5  |
| 2.2 | Roles y permisos                          | 5  |
| 2.3 | Direcciones de producción                 | 6  |
| 3   | Arquitectura                              | 7  |
| 3.1 | Vista general                             | 7  |
| 3.2 | Backend por capas                         | 7  |
| 3.3 | Ciclo de una petición                     | 8  |
| 3.4 | Frontend                                  | 8  |
| 3.5 | Arranque resistente                       | 9  |
| 4   | Tecnologías y estructura del proyecto     | 10 |
| 4.1 | Backend                                   | 10 |
| 4.2 | Frontend                                  | 10 |
| 4.3 | Datos, infraestructura y servicios        | 10 |
| 4.4 | Estructura del repositorio                | 11 |
| 5   | Modelo de datos                           | 12 |
| 5.1 | Mapa del modelo                           | 12 |
| 5.2 | Diccionario de datos                      | 12 |
| 5.3 | Migraciones, script SQL y datos iniciales | 17 |
| 6   | API REST                                  | 18 |
| 6.1 | Convenciones                              | 18 |
| 6.2 | Autenticación y sesión                    | 18 |
| 6.3 | Catálogo de endpoints                     | 19 |
| 7   | Seguridad                                 | 24 |
| 7.1 | Autenticación y control de acceso         | 24 |
| 7.2 | Contraseñas y recuperación                | 24 |
| 7.3 | Límites de peticiones                     | 24 |
| 7.4 | Cabeceras, CORS y otras defensas          | 25 |

- 8 Módulos funcionales** ..... 26
  - 8.1 Catálogo, cotización y precios ..... 26
  - 8.2 Reservas ..... 26
  - 8.3 Pagos ..... 26
  - 8.4 Ventas, facturas y reportes ..... 27
  - 8.5 Dashboard y estadísticas ..... 27
  - 8.6 PQR ..... 27
  - 8.7 Chatbot y recomendaciones con IA ..... 27
  - 8.8 Correos transaccionales ..... 28
  - 8.9 Paneles por rol ..... 28
- 9 Configuración** ..... 29
- 10 Instalación y ejecución local** ..... 32
  - 10.1 Requisitos ..... 32
  - 10.2 Backend ..... 32
  - 10.3 Frontend ..... 32
- 11 Pruebas** ..... 33
  - 11.1 Pruebas automatizadas del backend ..... 33
  - 11.2 Colección de Postman ..... 33
  - 11.3 Frontend ..... 34
- 12 Despliegue** ..... 35
  - 12.1 Railway: base de datos y backend ..... 35
  - 12.2 Cloudflare: frontend ..... 35
  - 12.3 Stripe ..... 35
  - 12.4 SendGrid ..... 35
  - 12.5 Verificación después de desplegar ..... 35
- 13 Operación y solución de problemas** ..... 37
  - 13.1 Mantenimiento ..... 37
- 14 Glosario** ..... 38

# 1 Introducción

Este manual describe cómo está construido Aurora Viajes y cómo se instala, configura, prueba, despliega y mantiene.

## 1.1 Propósito del documento

Reunir en un solo lugar la información técnica necesaria para entender, operar y hacer evolucionar la plataforma: su arquitectura, sus tecnologías, el modelo de datos, la API REST, las decisiones de seguridad y los procedimientos de instalación, pruebas y despliegue. Los datos de las tablas, rutas y variables de configuración se extrajeron directamente del código fuente, de modo que el manual refleja el sistema tal como está desplegado.

## 1.2 Alcance

Cubre el **frontend** (React + Vite), el **backend** (FastAPI), la **base de datos** (MySQL en producción y SQLite en desarrollo y pruebas), los **servicios externos** (Stripe, SendGrid y Groq) y la **infraestructura** (Railway, Cloudflare y GitHub). No es un manual de usuario final: el uso de las pantallas se menciona solo cuando ayuda a entender un flujo técnico.

## 1.3 Público objetivo

- **Desarrolladores** que mantengan o amplíen el sistema.
- **Administradores de la plataforma** que la desplieguen, configuren y vigilen.
- **Evaluadores técnicos** (instructores y auditores) que necesiten verificar cómo se cumplen los requisitos.

## 1.4 Convenciones

| Convención                                                                        | Significado                                                                                                                                           |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| texto monoespaciado                                                               | Código, rutas de archivos (relativas a la raíz del repositorio), comandos y endpoints.                                                                |
| MAYÚSCULAS                                                                        | Variables de entorno.                                                                                                                                 |
| <div><div>Pública</div><div>Sesión</div><div>Personal</div><div>Admin</div></div> | Nivel de acceso de un endpoint: sin sesión, con cualquier sesión (sobre sus propios datos), personal (empleado o administrador) y solo administrador. |
| Montos                                                                            | Pesos colombianos (COP) con dos decimales. Stripe los recibe en centavos.                                                                             |
| Fechas                                                                            | ISO 8601. Los instantes se guardan en UTC y se muestran en hora de Colombia (UTC-5).                                                                  |

## 1.5 Documentos relacionados

| Documento  | Contenido                                                       |
|------------|-----------------------------------------------------------------|
| README .md | Presentación del proyecto, puesta en marcha rápida y variables. |

| Documento                    | Contenido                                                                  |
|------------------------------|----------------------------------------------------------------------------|
| DESPLIEGUE.md                | Guía de despliegue en Railway y Cloudflare, costos y problemas frecuentes. |
| INTEGRACION_FULL_STACK.md    | Cómo se conectan el frontend, la API y la base.                            |
| backend/sql/schema.sql       | Script SQL de la estructura, generado desde los modelos.                   |
| backend/postman/collections/ | Colección de Postman con las peticiones del Quinto Avance.                 |

## 2 Descripción general del sistema

Aurora Viajes es una agencia de viajes en línea: el cliente arma su viaje, ve el precio final en cada paso, paga en línea y recibe la confirmación con su factura.

### 2.1 Qué hace la plataforma

**Para el cliente**

- Catálogo de 10 destinos con precio desde.
- Paquete cerrado o viaje a la carta: vuelo de ida y regreso, hotel y excursiones.
- Cotización exacta antes de confirmar.
- Pago con tarjeta por Stripe y factura en PDF.
- Panel propio: reservas, facturas, PQR y chatbot.
- Recomendaciones de destinos con IA.

**Para el personal**

- Reservas a nombre de clientes y cobro en mostrador.
- Gestión de reservas: confirmar, editar y cancelar.
- Vuelos y manifiesto de pasajeros.
- Ventas, facturas y reportes en PDF y Excel.
- Dashboard con indicadores, gráficos y filtros.
- Atención de PQR.

**Para el administrador**

- Catálogo completo: destinos, hoteles, excursiones, paquetes, aerolíneas, ciudades, productos y servicios.
- Programación de vuelos.
- Usuarios y roles.
- Mensajes del formulario de contacto.

**Transversal**

- Correos: bienvenida, recuperación de contraseña, acuse de reserva y confirmación de pago.
- Seguridad por roles en el servidor.
- Arranque resistente sin base de datos.
- Migraciones automáticas del esquema.

### 2.2 Roles y permisos

Hay tres roles. El **registro público solo crea clientes**, aunque la petición intente enviar otro rol. Al personal lo crea el administrador con una clave provisional que debe cambiarse al primer acceso. El rol de cada petición se lee de la base de datos, no del token.

| Capacidad                                                     | Cliente | Empleado | Administrador |
|---------------------------------------------------------------|---------|----------|---------------|
| Ver el catálogo y cotizar                                     | •       | •        | •             |
| Reservar y pagar sus propias reservas                         | •       | •        | •             |
| Ver sus facturas, registrar PQR, usar el chatbot              | •       | •        | •             |
| Reservar a nombre de un cliente y cobrar en el mostrador      | —       | •        | •             |
| Editar, confirmar y cancelar reservas (incluidas las pagadas) | —       | •        | •             |

| Capacidad                                    | Cliente | Empleado | Administrador |
|----------------------------------------------|---------|----------|---------------|
| Ventas, reportes, dashboard y gestión de PQR | —       | ●        | ●             |
| Consultar vuelos y descargar manifiestos     | —       | ●        | ●             |
| Programar vuelos y gestionar el catálogo     | —       | —        | ●             |
| Usuarios, roles y mensajes de contacto       | —       | —        | ●             |

Permisos sembrados: administrador — usuarios:gestionar, productos:gestionar, servicios:gestionar, reservas:gestionar, mensajes:leer; empleado — reservas:gestionar; cliente — reservas:crear.

## 2.3 Direcciones de producción

| Componente                | Dirección                                                                                                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Aplicación web (frontend) | <a href="https://aurora-viajes.carousel-app.workers.dev">https://aurora-viajes.carousel-app.workers.dev</a>                                     |
| API REST (backend)        | <a href="https://aurora-backend-production-a2b4.up.railway.app/api">https://aurora-backend-production-a2b4.up.railway.app/api</a>               |
| Estado de la API          | <a href="https://aurora-backend-production-a2b4.up.railway.app/api/health">https://aurora-backend-production-a2b4.up.railway.app/api/health</a> |
| Repositorio               | <a href="https://github.com/davidabb2000/aurora-viajes">https://github.com/davidabb2000/aurora-viajes</a> (rama main)                           |

# 3 Arquitectura

Arquitectura cliente-servidor de tres capas: una aplicación de una sola página (SPA) en Cloudflare consume una API REST en Railway, que persiste en MySQL.

## 3.1 Vista general

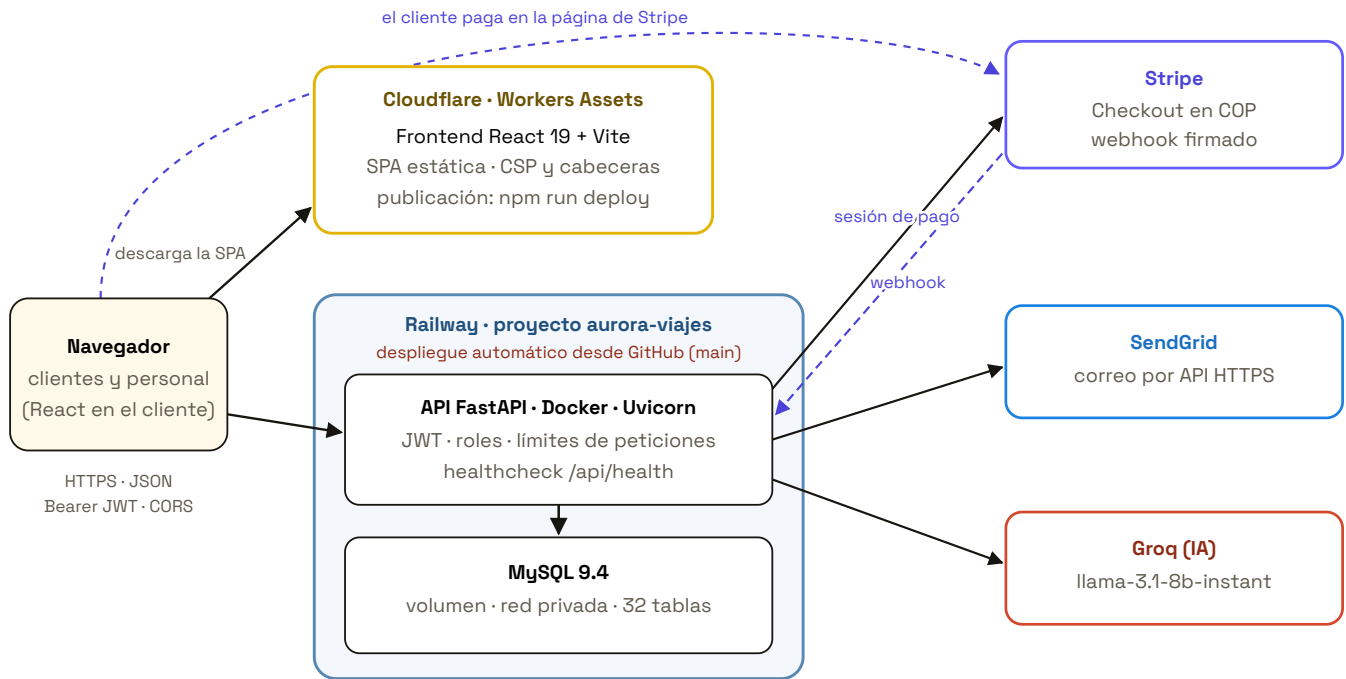


Figura 1. Componentes y comunicación en producción.

- El navegador descarga la SPA desde Cloudflare y, a partir de ahí, habla con la API por HTTPS con JSON y el token JWT en la cabecera Authorization. CORS solo admite el origen del frontend.
- La API corre en un contenedor Docker en Railway y se conecta a MySQL por la red privada del proyecto (la base no está expuesta a internet).
- Los servicios externos se usan por HTTPS: Stripe (sesiones de pago y webhook firmado), SendGrid (correo transaccional) y Groq (IA compatible con la API de OpenAI).
- El backend se despliega solo al hacer *push* a main; el frontend se publica con `npm run deploy` (Wrangler).

## 3.2 Backend por capas

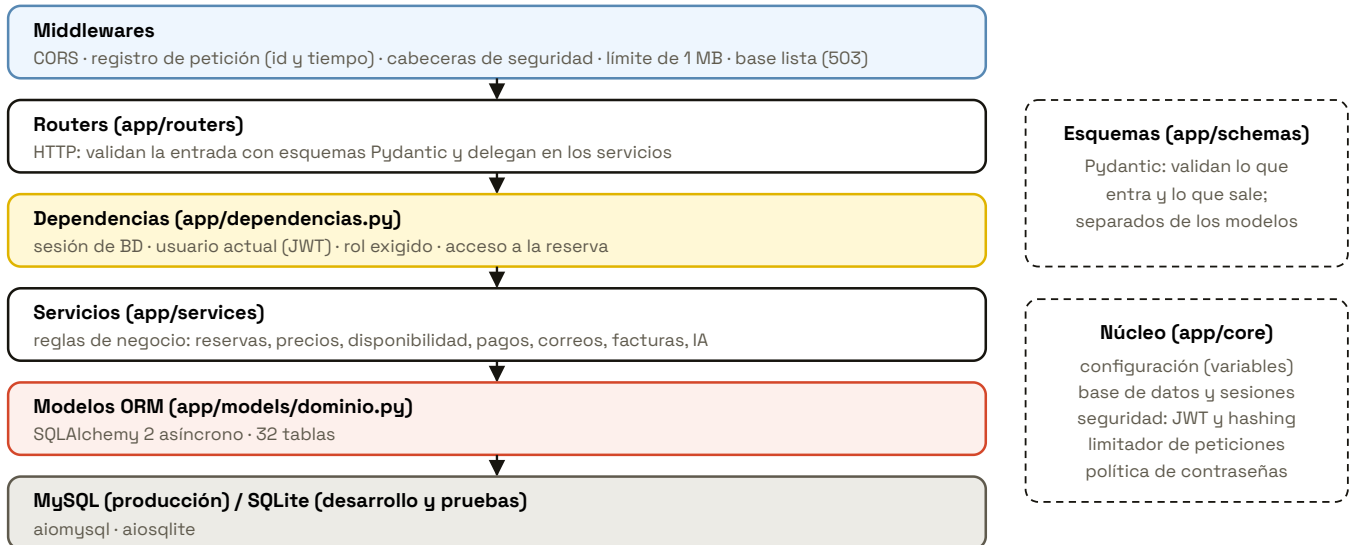


Figura 2. Capas del backend y dependencias.

Cada capa solo conoce a la siguiente: los *routers* no contienen reglas de negocio y los servicios no saben de HTTP. Los errores de dominio (app/errores.py) se traducen a respuestas HTTP con un formato uniforme en app/main.py. Las tareas lentas (correos) se ejecutan en segundo plano después de responder.

## 3.3 Ciclo de una petición

1. **CORS:** comprueba el origen.
2. **Registro:** asigna un identificador (X-Peticion-Id) y mide la duración (X-Tiempo-Respuesta-ms).
3. **Cabeceras de seguridad:** CSP, nosniff, DENY de marcos, HSTS en HTTPS.
4. **Límite de tamaño:** más de 1 MB responde 413.
5. **Base lista:** si la base aún no está disponible, 503 con Retry-After.
6. **Dependencias:** sesión de base de datos, usuario del token (con su rol leído de la base) y permiso exigido por la ruta.
7. **Router y servicio:** validación con Pydantic, reglas de negocio y *commit*.
8. **Respuesta** JSON (o PDF, Excel, CSV) y tareas en segundo plano.

## 3.4 Frontend

- **SPA con React 19 y React Router 7;** Cloudflare sirve index.html para cualquier ruta (not\_found\_handling: single-page-application).
- **Dos diseños:** ClientLayout (sitio público con cabecera y pie) y AdminLayout (panel con barra lateral), elegidos por la ruta /panel\*. La barra lateral y las pestañas se adaptan al rol.
- **Sesión:** AuthContext guarda el token y el usuario en sessionStorage (o en localStorage con «Recordarme») y la sincroniza entre pestañas.

- **Acceso a la API:** `utils/api.js` (solicitar: URL base de `VITE_API_URL`, token y errores) y el hook `useCarga` (carga, recarga y errores sin parpadeo).
- **Estilos:** Tailwind CSS 4 con el sistema de diseño «Cielo y papel» (tokens de color, vidrio esmerilado) y tipografías Space Grotesk, DM Serif Display y JetBrains Mono.
- **Protección de rutas:** `RutaProtegida` (exige sesión y, si toca, obliga a cambiar la clave provisional) y `SoloInvitados`. Es solo experiencia de uso: la seguridad real está en el backend.

### 3.5 Arranque resistente

La API arranca aunque la base no responda. Mientras tanto `GET /api/health` informa el estado en `baseDeDatos` (lista, sin conexion, migracion pendiente o error), el resto de rutas responde 503 con CORS y el servicio reintentará con espera creciente (de 5 a 60 segundos). Cuando la base vuelve, crea las tablas que falten, migra el esquema y siembra los datos iniciales sin redesplegar.

## 4 Tecnologías y estructura del proyecto

Versiones fijadas en requirements.txt y package.json, y organización del repositorio.

### 4.1 Backend

| Tecnología                   | Versión              | Uso                                       |
|------------------------------|----------------------|-------------------------------------------|
| Python                       | 3.12                 | Lenguaje; imagen Docker python:3.12-slim. |
| FastAPI                      | 0.141.1              | Framework web y validación.               |
| Starlette · Uvicorn          | 1.6.0 · 0.52.4       | Base ASGI y servidor.                     |
| SQLAlchemy                   | 2.0.52               | ORM asíncrono.                            |
| aiomysql · aiosqlite         | 0.2.0 · 0.20.0       | Controladores de MySQL y SQLite.          |
| Pydantic · pydantic-settings | 2.13.5 · 2.15.0      | Esquemas y configuración.                 |
| PyJWT                        | 2.10.1               | Tokens de sesión (HS256).                 |
| pwdlib (Argon2)              | 0.2.1                | Hash de contraseñas.                      |
| httpx                        | 0.28.1               | Cliente HTTP (Stripe, SendGrid, Groq).    |
| aiosmtplib                   | 3.0.1                | SMTP (alternativa fuera de Railway).      |
| pytest                       | requirements-dev.txt | Pruebas automatizadas.                    |

### 4.2 Frontend

| Tecnología                         | Versión      | Uso                                                                 |
|------------------------------------|--------------|---------------------------------------------------------------------|
| React · React DOM                  | 19.2         | Interfaz.                                                           |
| React Router DOM                   | 7.18         | Rutas de la SPA.                                                    |
| Vite                               | 8.2          | Servidor de desarrollo y compilación.                               |
| Tailwind CSS                       | 4.3          | Estilos.                                                            |
| @cloudflare/vite-plugin · Wrangler | 1.56 · 4.135 | Compilación y publicación en Cloudflare.                            |
| oxlint                             | 1.75         | Análisis estático (lint).                                           |
| Fontsource                         | 5.3          | Tipografías autoalojadas (sin Google Fonts en tiempo de ejecución). |

### 4.3 Datos, infraestructura y servicios

| Servicio            | Uso                                                   |
|---------------------|-------------------------------------------------------|
| MySQL 9.4 (Railway) | Base de producción, con volumen propio y red privada. |

| Servicio                  | Uso                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------|
| SQLite                    | Desarrollo local y pruebas automáticas.                                                  |
| Railway                   | Contenedor del backend (Dockerfile, healthcheck, reinicio ante fallos) y servicio MySQL. |
| Cloudflare Workers Assets | Hospedaje de la SPA con CSP y cabeceras de seguridad.                                    |
| GitHub                    | Repositorio público; cada push a main despliega el backend.                              |
| Stripe                    | Cobros con Checkout en COP y webhook firmado (modo de pruebas).                          |
| SendGrid                  | Correo transaccional por API HTTPS (v3).                                                 |
| Groq                      | Modelo llama-3.1-8b-instant por una API compatible con OpenAI.                           |

## 4.4 Estructura del repositorio

Un solo repositorio con dos proyectos: backend/ y frontend/.

```

aurora-viajes/
├── backend/
│   ├── app/
│   │   ├── core/           # configuración, BD, JWT y hash, limitador, política de claves
│   │   ├── models/        # dominio.py: los 32 modelos ORM
│   │   ├── routers/       # 11 routers: auth, usuarios, catálogo, reservas, pagos, comercial...
│   │   ├── schemas/       # esquemas Pydantic de entrada y salida
│   │   ├── services/      # reglas de negocio: reservas, precios, pagos, correos, IA...
│   │   ├── dependencias.py · errores.py · middlewares.py · main.py
│   ├── scripts/           # desarrollo_local, exportar_esquema, probar_correo
│   ├── sql/schema.sql     # estructura de la base, generada desde los modelos
│   ├── tests/             # 16 archivos de prueba + conftest.py + utiles.py
│   ├── postman/           # colección de Postman
│   └── Dockerfile · railway.json · requirements*.txt · pyproject.toml · .env.example
├── frontend/
│   ├── src/
│   │   ├── components/    # cabecera, pie, barra lateral, asistente de reserva...
│   │   ├── context/       # AuthContext: sesión y rol
│   │   ├── layouts/       # ClientLayout y AdminLayout
│   │   ├── pages/         # sitio público, cuenta, pago, panel y Quinto Avance
│   │   ├── utils/         # api.js, useCarga, rutas, validaciones
│   │   └── data/ · App.jsx · main.jsx · index.css
│   ├── index.html · vite.config.js · wrangler.jsonc · package.json
└── README.md · DESPLIEGUE.md · INTEGRACION_FULL_STACK.md

```

## 5 Modelo de datos

32 tablas normalizadas a tercera forma normal (3FN). Los estados y métodos son tablas de catálogo, los montos usan DECIMAL(12,2) y los instantes se guardan en UTC.

### 5.1 Mapa del modelo

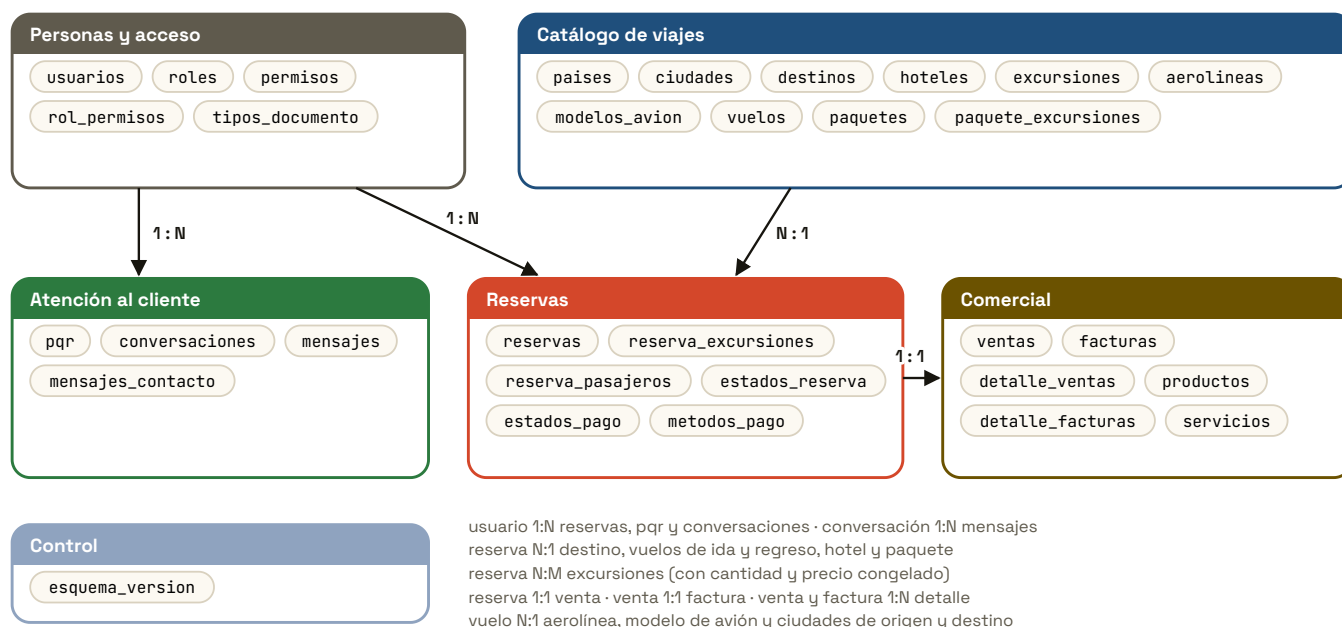


Figura 3. Grupos de tablas y relaciones principales. El detalle de columnas y llaves está en el diccionario de datos.

- Cada **reserva** guarda su desglose (vuelo, hotel y excursiones) y las tarifas vigentes al reservar: el total nunca depende de precios que cambien después. Una restricción CHECK garantiza que el total sea la suma del desglose.
- Cada reserva crea en la misma transacción su **venta** (pendiente) y su **factura** (emitida). Pagar la completa; cancelar la anula.
- La numeración de facturas es AUR-AAAAMDD-NNNNNN (fecha de emisión y número de venta).

### 5.2 Diccionario de datos

Convenciones: **PK** llave primaria, **FK** llave foránea (tabla.columna), UQ valor único; «nulo» indica si admite NULL.

### aerolineas

Aerolíneas (código y nombre).

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| codigo | varchar(4)  | UQ | no |
| nombre | varchar(80) | UQ | no |

### ciudades

Ciudades, cada una con su país.

|         |             |              |    |
|---------|-------------|--------------|----|
| id      | integer     | PK           | no |
| pais_id | integer     | FK→países.id | no |
| nombre  | varchar(80) |              | no |

### conversaciones

Conversaciones del chatbot, una por hilo del cliente.

|                |          |                |    |
|----------------|----------|----------------|----|
| id             | integer  | PK             | no |
| cliente_id     | integer  | FK→usuarios.id | sí |
| creado_en      | datetime |                | no |
| actualizado_en | datetime |                | no |

### destinos

Destinos ofrecidos: ciudad, descripción, precio base e imagen.

|             |                |                      |    |
|-------------|----------------|----------------------|----|
| id          | integer        | PK                   | no |
| ciudad_id   | integer        | FK→ciudades.id<br>UQ | no |
| descripcion | text           |                      | sí |
| precio_base | numeric(12, 2) |                      | no |
| imagen_slug | varchar(80)    |                      | sí |
| activo      | boolean        |                      | no |

### detalle\_facturas

Líneas de cada factura.

|                 |                |                |    |
|-----------------|----------------|----------------|----|
| id              | integer        | PK             | no |
| factura_id      | integer        | FK→facturas.id | no |
| nombre          | varchar(140)   |                | no |
| cantidad        | integer        |                | no |
| precio_unitario | numeric(12, 2) |                | no |
| subtotal        | numeric(12, 2) |                | no |

### detalle\_ventas

Líneas de cada venta (producto, servicio o concepto del viaje).

|                 |                |                 |    |
|-----------------|----------------|-----------------|----|
| id              | integer        | PK              | no |
| venta_id        | integer        | FK→ventas.id    | no |
| producto_id     | integer        | FK→productos.id | sí |
| servicio_id     | integer        | FK→servicios.id | sí |
| nombre          | varchar(140)   |                 | no |
| cantidad        | integer        |                 | no |
| precio_unitario | numeric(12, 2) |                 | no |
| subtotal        | numeric(12, 2) |                 | no |

### esquema\_version

Versión del esquema aplicada (control de migraciones).

|             |          |    |    |
|-------------|----------|----|----|
| version     | integer  | PK | no |
| aplicada_en | datetime |    | no |

### estados\_pago

Catálogo: pendiente, pagado, fallido.

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| codigo | varchar(30) | UQ | no |
| nombre | varchar(40) |    | no |

### estados\_reserva

Catálogo: pendiente, confirmada, cancelada.

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| codigo | varchar(30) | UQ | no |
| nombre | varchar(40) |    | no |

### excursiones

Excursiones por ciudad, con duración y precio.

|                |                |                |    |
|----------------|----------------|----------------|----|
| id             | integer        | PK             | no |
| nombre         | varchar(120)   |                | no |
| ciudad_id      | integer        | FK→ciudades.id | no |
| duracion_horas | integer        |                | no |
| precio         | numeric(12, 2) |                | no |
| descripcion    | text           |                | sí |
| activo         | boolean        |                | no |

### facturas

Factura de cada venta: número AUR-AAAAMMDD-NNNNNN y estado.

|           |             |                 |    |
|-----------|-------------|-----------------|----|
| id        | integer     | PK              | no |
| venta_id  | integer     | FK→ventas.id UQ | no |
| numero    | varchar(40) | UQ              | no |
| estado    | varchar(30) |                 | no |
| creado_en | datetime    |                 | no |

### hoteles

Hoteles por ciudad, con estrellas y precio por noche.

|              |                |                |    |
|--------------|----------------|----------------|----|
| id           | integer        | PK             | no |
| nombre       | varchar(120)   |                | no |
| ciudad_id    | integer        | FK→ciudades.id | no |
| estrellas    | integer        |                | no |
| precio_noche | numeric(12, 2) |                | no |
| descripcion  | text           |                | sí |
| activo       | boolean        |                | no |

### mensajes

Mensajes de cada conversación del chatbot (usuario o asistente).

|                 |             |                      |    |
|-----------------|-------------|----------------------|----|
| id              | integer     | PK                   | no |
| conversacion_id | integer     | FK→conversaciones.id | no |
| rol             | varchar(20) |                      | no |
| contenido       | text        |                      | no |
| creado_en       | datetime    |                      | no |

### mensajes\_contacto

Mensajes del formulario de contacto.

|           |              |    |    |
|-----------|--------------|----|----|
| id        | integer      | PK | no |
| nombre    | varchar(80)  |    | no |
| correo    | varchar(100) |    | no |
| mensaje   | text         |    | no |
| creado_en | datetime     |    | no |

### metodos\_pago

Catálogo: stripe, transferencia, efectivo, tarjeta (datáfono).

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| codigo | varchar(30) | UQ | no |
| nombre | varchar(40) |    | no |

### modelos\_avion

Modelos de avión y su capacidad física.

|           |             |    |    |
|-----------|-------------|----|----|
| id        | integer     | PK | no |
| nombre    | varchar(80) | UQ | no |
| capacidad | integer     |    | no |

### países

Países.

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| nombre | varchar(80) | UQ | no |

### paquete\_excursiones

Excursiones incluidas en cada paquete (N:M).

|              |         |                         |    |
|--------------|---------|-------------------------|----|
| paquete_id   | integer | PK<br>FK→paquetes.id    | no |
| excursion_id | integer | PK<br>FK→excursiones.id | no |

### paquetes

Paquetes cerrados: vuelos de ida y regreso, hotel y precio base.

|                  |                |                |    |
|------------------|----------------|----------------|----|
| id               | integer        | PK             | no |
| nombre           | varchar(140)   |                | no |
| destino_id       | integer        | FK→destinos.id | no |
| vuelo_id         | integer        | FK→vuelos.id   | no |
| vuelo_regreso_id | integer        | FK→vuelos.id   | sí |
| hotel_id         | integer        | FK→hoteles.id  | no |
| noches           | integer        |                | no |
| precio_base      | numeric(12, 2) |                | no |
| activo           | boolean        |                | no |

### permisos

Permisos del sistema.

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| nombre | varchar(80) | UQ | no |

## pqr

Peticiones, quejas y reclamos, con estado y respuesta.

|                |              |                |    |
|----------------|--------------|----------------|----|
| id             | integer      | PK             | no |
| cliente_id     | integer      | FK→usuarios.id | no |
| tipo           | varchar(30)  |                | no |
| asunto         | varchar(140) |                | no |
| descripcion    | text         |                | no |
| respuesta      | text         |                | sí |
| estado         | varchar(30)  |                | no |
| creado_en      | datetime     |                | no |
| actualizado_en | datetime     |                | no |

## productos

Productos vendibles en el mostrador (p. ej. seguros).

|             |                |    |    |
|-------------|----------------|----|----|
| id          | integer        | PK | no |
| nombre      | varchar(100)   |    | no |
| descripcion | text           |    | sí |
| precio      | numeric(12, 2) |    | no |
| activo      | boolean        |    | no |

## reserva\_excursiones

Excursiones de una reserva, con cantidad y precio congelado.

|                 |                |                      |    |
|-----------------|----------------|----------------------|----|
| reserva_id      | integer        | PK FK→reservas.id    | no |
| excursion_id    | integer        | PK FK→excursiones.id | no |
| cantidad        | integer        |                      | no |
| precio_unitario | numeric(12, 2) |                      | no |

## reserva\_pasajeros

Datos de los pasajeros de una reserva.

|                   |             |                       |    |
|-------------------|-------------|-----------------------|----|
| id                | integer     | PK                    | no |
| reserva_id        | integer     | FK→reservas.id        | no |
| nombre            | varchar(40) |                       | no |
| apellido          | varchar(40) |                       | no |
| tipo_documento_id | integer     | FK→tipos_documento.id | no |
| numero_documento  | varchar(20) |                       | no |

## reservas

Reservas: cliente, destino, vuelos, hotel o paquete, fechas, pasajeros, estado, pago y montos congelados.

|                        |                |                       |    |
|------------------------|----------------|-----------------------|----|
| id                     | integer        | PK                    | no |
| usuario_id             | integer        | FK→usuarios.id        | no |
| creada_por_id          | integer        | FK→usuarios.id        | sí |
| destino_id             | integer        | FK→destinos.id        | no |
| vuelo_id               | integer        | FK→vuelos.id          | sí |
| vuelo_regreso_id       | integer        | FK→vuelos.id          | sí |
| paquete_id             | integer        | FK→paquetes.id        | sí |
| hotel_id               | integer        | FK→hoteles.id         | sí |
| fecha_salida           | date           |                       | no |
| fecha_regreso          | date           |                       | no |
| pasajeros              | integer        |                       | no |
| telefono_contacto      | varchar(10)    |                       | no |
| notas                  | varchar(300)   |                       | sí |
| estado_id              | integer        | FK→estados_reserva.id | no |
| estado_pago_id         | integer        | FK→estados_pago.id    | no |
| metodo_pago_id         | integer        | FK→metodos_pago.id    | sí |
| monto_total            | numeric(12, 2) |                       | no |
| monto_vuelo            | numeric(12, 2) |                       | no |
| monto_hotel            | numeric(12, 2) |                       | no |
| monto_excursiones      | numeric(12, 2) |                       | no |
| stripe_session_id      | varchar(255)   |                       | sí |
| pago_referencia        | varchar(80)    |                       | sí |
| pago_registrado_por_id | integer        | FK→usuarios.id        | sí |
| pagado_en              | datetime       |                       | sí |
| creado_en              | datetime       |                       | no |
| actualizado_en         | datetime       |                       | no |

## rol\_permisos

Relación N:M entre roles y permisos.

|            |         |                   |    |
|------------|---------|-------------------|----|
| rol_id     | integer | PK FK→roles.id    | no |
| permiso_id | integer | PK FK→permisos.id | no |

### roles

Roles: administrador, empleado y cliente.

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| nombre | varchar(30) | UQ | no |

### servicios

Servicios vendibles en el mostrador (p. ej. traslados).

|             |                |    |    |
|-------------|----------------|----|----|
| id          | integer        | PK | no |
| nombre      | varchar(100)   |    | no |
| descripcion | text           |    | sí |
| precio      | numeric(12, 2) |    | no |
| activo      | boolean        |    | no |

### tipos\_documento

Catálogo: CC, TI, CE y PA.

|        |             |    |    |
|--------|-------------|----|----|
| id     | integer     | PK | no |
| codigo | varchar(5)  | UQ | no |
| nombre | varchar(40) |    | no |

### usuarios

Cuentas de clientes y personal: hash de la contraseña, rol, estado y versión de sesión.

|                         |              |                       |    |
|-------------------------|--------------|-----------------------|----|
| id                      | integer      | PK                    | no |
| nombre                  | varchar(40)  |                       | no |
| apellido                | varchar(40)  |                       | no |
| tipo_documento_id       | integer      | FK→tipos_documento.id | no |
| numero_documento        | varchar(12)  | UQ                    | no |
| direccion               | varchar(80)  |                       | no |
| telefono                | varchar(10)  |                       | no |
| correo                  | varchar(60)  | UQ                    | no |
| contrasena_hash         | varchar(255) |                       | no |
| activo                  | boolean      |                       | no |
| rol_id                  | integer      | FK→roles.id           | no |
| sesion_version          | integer      |                       | no |
| debe_cambiar_contraseña | boolean      |                       | no |
| acepto_datos_personales | datetime     |                       | sí |
| creado_en               | datetime     |                       | no |
| actualizado_en          | datetime     |                       | no |

### ventas

Venta de cada reserva o de mostrador: subtotal, descuento, impuestos, total y estado.

|            |                |                      |    |
|------------|----------------|----------------------|----|
| id         | integer        | PK                   | no |
| cliente_id | integer        | FK→usuarios.id       | no |
| usuario_id | integer        | FK→usuarios.id       | sí |
| reserva_id | integer        | FK→reservas.id<br>UQ | sí |
| subtotal   | numeric(12, 2) |                      | no |
| descuento  | numeric(12, 2) |                      | no |
| impuestos  | numeric(12, 2) |                      | no |
| total      | numeric(12, 2) |                      | no |
| estado     | varchar(30)    |                      | no |
| creado_en  | datetime       |                      | no |

## vuelos

Programación de vuelos: aerolínea, avión, ruta, horarios, capacidad y estado.

|                  |             |                     |    |
|------------------|-------------|---------------------|----|
| id               | integer     | PK                  | no |
| numero_vuelo     | varchar(20) |                     | no |
| aerolinea_id     | integer     | FK→aerolineas.id    | no |
| modelo_avion_id  | integer     | FK→modelos_avion.id | no |
| origen_id        | integer     | FK→ciudades.id      | no |
| destino_id       | integer     | FK→ciudades.id      | no |
| fecha_salida     | datetime    |                     | no |
| fecha_llegada    | datetime    |                     | no |
| capacidad_maxima | integer     |                     | no |
| puerta           | varchar(10) |                     | sí |
| terminal         | varchar(20) |                     | sí |
| estado           | varchar(20) |                     | no |
| activo           | boolean     |                     | no |

## 5.3 Migraciones, script SQL y datos iniciales

- **Creación:** al arrancar, create\_all crea las tablas que faltan.
- **Migraciones** (services/migraciones.py): llevan una base antigua al esquema actual. Son idempotentes, siguen el patrón «expandir, copiar, contraer» y guardan antes una copia de los datos en tablas respaldo\_v2\_\*. MIGRACION\_AUTOMATICA=false las pausa para hacer un respaldo manual; la versión aplicada queda en esquema\_version.
- **Script SQL:** python -m scripts.exportar\_esquema regenera backend/sql/schema.sql desde los modelos; así nunca se desfasa del código.
- **Siembra:** la primera vez se cargan roles y permisos, tipos de documento, estados y métodos de pago, aerolíneas, modelos de avión, países y ciudades, 10 destinos con hoteles y excursiones, paquetes «Aurora» y la programación de vuelos (que se renueva sola), además del administrador inicial.

## 6 API REST

84 endpoints bajo /api, más los de estado. Todos intercambian JSON salvo las descargas (PDF, Excel y CSV).

### 6.1 Convenciones

| Aspecto                   | Detalle                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| URL base                  | Producción: <code>https://aurora-backend-production-a2b4.up.railway.app/api</code> . Desarrollo: <code>http://127.0.0.1:8001/api</code> (o /api a través del proxy de Vite). |
| Autenticación             | Authorization: Bearer <token>, obtenido en <code>POST /api/auth/login</code> .                                                                                               |
| Estado                    | <code>GET /api/health</code> → <code>{"estado": "ok", "baseDeDatos": "lista"}</code> .                                                                                       |
| Documentación interactiva | /docs solo con <code>DEPURACION=true</code> (desarrollo). En producción responde 404.                                                                                        |
| Errores                   | Formato único: <code>{"codigo", "mensaje", "ruta", "detalles"}</code> .                                                                                                      |

| HTTP | código                                            | Cuándo                                                                        |
|------|---------------------------------------------------|-------------------------------------------------------------------------------|
| 400  | error_de_dominio                                  | Regla de negocio incumplida (p. ej., firma de webhook inválida).              |
| 401  | no_autenticado                                    | Sin token, token vencido o sesión cerrada.                                    |
| 403  | permiso_denegado · cambio_de_contrasena_requerido | El rol no alcanza, el recurso no es suyo o debe cambiar la clave provisional. |
| 404  | recurso_no_encontrado                             | No existe.                                                                    |
| 409  | conflicto_de_negocio · conflicto_de_datos         | Choca con el estado actual o con datos existentes (duplicados).               |
| 413  | —                                                 | Cuerpo mayor que TAMANO_MAXIMO_CUERPO.                                        |
| 422  | datos_invalidos                                   | Validación de Pydantic; detalles lista el problema por campo.                 |
| 429  | demasiadas_peticiones                             | Límite de peticiones superado; incluye Retry-After.                           |
| 503  | servicio_no_disponible                            | Base no disponible o servicio externo sin configurar.                         |
| 500  | error_interno                                     | Error inesperado (queda en el log con su identificador).                      |

### 6.2 Autenticación y sesión

- El login devuelve un **JWT HS256** firmado con SECRET\_KEY, válido MINUTOS\_EXPIRACION\_TOKEN (60 por defecto), con sub, rol, purpose, sv (versión de sesión), jti, iat y exp. El algoritmo está fijado: se rechaza alg: none.
- En cada petición el usuario y su **rol se leen de la base**: cambiar un rol surte efecto de inmediato y un token manipulado no concede permisos.
- Cambiar la contraseña o «cerrar sesiones» incrementa session\_version e invalida los tokens anteriores.
- Una cuenta con clave provisional solo puede cambiarla (403 cambio\_de\_contrasena\_requerido en lo demás).

- Recuperación: token con purpose=recuperacion, válido 60 minutos, con la huella del hash vigente; al cambiar la clave la huella deja de coincidir y el enlace no sirve una segunda vez.

## 6.3 Catálogo de endpoints

Nivel de acceso según la dependencia exigida en el código: **Pública** **Sesión** **Personal** **Admin**. Con «Sesión», el servidor limita los datos a los del propio usuario.

### Autenticación y sesión · app/routers/auth.py

| Método | Ruta                         | Acceso  | Descripción                                                                                                 |
|--------|------------------------------|---------|-------------------------------------------------------------------------------------------------------------|
| POST   | /api/auth/cambiar-contrasena | Sesión  | Cambia la contraseña (obligatorio con clave provisional); emite un token nuevo y cierra las demás sesiones. |
| POST   | /api/auth/cerrar-sesiones    | Sesión  | Invalida todos los tokens emitidos a la cuenta.                                                             |
| POST   | /api/auth/login              | Pública | Inicia sesión con correo y contraseña; devuelve el token JWT y los datos de la cuenta.                      |
| POST   | /api/auth/recuperar          | Pública | Envía el enlace de recuperación (1 hora, un solo uso) si la cuenta existe; responde igual en ambos casos.   |
| POST   | /api/auth/restablecer        | Pública | Fija una contraseña nueva con el token del enlace de recuperación.                                          |
| GET    | /api/auth/yo                 | Sesión  | Datos de la cuenta de la sesión.                                                                            |

### Usuarios · app/routers/usuarios.py

| Método | Ruta                              | Acceso  | Descripción                                                                            |
|--------|-----------------------------------|---------|----------------------------------------------------------------------------------------|
| GET    | /api/usuarios                     | Admin   | Lista todas las cuentas.                                                               |
| POST   | /api/usuarios                     | Admin   | Crea una cuenta con rol y clave provisional.                                           |
| POST   | /api/usuarios/registro            | Pública | Registro público: siempre crea una cuenta de cliente y envía el correo de bienvenida.  |
| GET    | /api/usuarios/{usuario_id}        | Admin   | Detalle de una cuenta.                                                                 |
| PUT    | /api/usuarios/{usuario_id}        | Admin   | Edita datos, correo o rol (debe quedar un administrador; nadie cambia su propio rol).  |
| DELETE | /api/usuarios/{usuario_id}        | Admin   | Elimina una cuenta sin historial; con reservas, ventas o PQR se desactiva en su lugar. |
| PATCH  | /api/usuarios/{usuario_id}/estado | Admin   | Activa o desactiva una cuenta.                                                         |

### Clientes (mostrador) · app/routers/clientes.py

| Método | Ruta          | Acceso   | Descripción                                                                      |
|--------|---------------|----------|----------------------------------------------------------------------------------|
| GET    | /api/clientes | Personal | Busca clientes activos por nombre, correo o documento para reservar a su nombre. |
| POST   | /api/clientes | Personal | Alta rápida de un cliente en el mostrador.                                       |

## Catálogo · app/routers/catalogo.py

| Método | Ruta                                          | Acceso   | Descripción                                                             |
|--------|-----------------------------------------------|----------|-------------------------------------------------------------------------|
| GET    | /api/aerolineas                               | Personal | Aerolíneas.                                                             |
| POST   | /api/aerolineas                               | Admin    | Crea una aerolínea.                                                     |
| GET    | /api/catalogos/destinos                       | Pública  | Destinos activos con su precio desde (portada).                         |
| GET    | /api/catalogos/destinos/{destino_id}/opciones | Pública  | Vuelos de ida y regreso, hoteles, excursiones y paquetes de un destino. |
| GET    | /api/ciudades                                 | Personal | Ciudades.                                                               |
| POST   | /api/ciudades                                 | Admin    | Crea una ciudad.                                                        |
| GET    | /api/destinos                                 | Personal | Todos los destinos, también los inactivos.                              |
| POST   | /api/destinos                                 | Admin    | Crea un destino.                                                        |
| PUT    | /api/destinos/{destino_id}                    | Admin    | Edita un destino.                                                       |
| DELETE | /api/destinos/{destino_id}                    | Admin    | Retira un destino del catálogo (lo desactiva).                          |
| GET    | /api/modelos-avion                            | Personal | Modelos de avión.                                                       |
| POST   | /api/modelos-avion                            | Admin    | Crea un modelo de avión.                                                |
| GET    | /api/productos                                | Pública  | Productos.                                                              |
| POST   | /api/productos                                | Admin    | Crea un producto.                                                       |
| GET    | /api/productos/{producto_id}                  | Pública  | Detalle de un producto.                                                 |
| PUT    | /api/productos/{producto_id}                  | Admin    | Edita un producto.                                                      |
| DELETE | /api/productos/{producto_id}                  | Admin    | Elimina un producto.                                                    |
| GET    | /api/servicios                                | Pública  | Servicios.                                                              |
| POST   | /api/servicios                                | Admin    | Crea un servicio.                                                       |
| GET    | /api/servicios/{servicio_id}                  | Pública  | Detalle de un servicio.                                                 |
| PUT    | /api/servicios/{servicio_id}                  | Admin    | Edita un servicio.                                                      |
| DELETE | /api/servicios/{servicio_id}                  | Admin    | Elimina un servicio.                                                    |

## Vuelos, hoteles, excursiones y paquetes · app/routers/viajes.py

| Método | Ruta                            | Acceso   | Descripción            |
|--------|---------------------------------|----------|------------------------|
| GET    | /api/excursiones                | Personal | Excursiones.           |
| POST   | /api/excursiones                | Admin    | Crea una excursión.    |
| PUT    | /api/excursiones/{excursion_id} | Admin    | Edita una excursión.   |
| DELETE | /api/excursiones/{excursion_id} | Admin    | Elimina una excursión. |
| GET    | /api/hoteles                    | Personal | Hoteles.               |
| POST   | /api/hoteles                    | Admin    | Crea un hotel.         |

| Método | Ruta                       | Acceso   | Descripción                                                                       |
|--------|----------------------------|----------|-----------------------------------------------------------------------------------|
| PUT    | /api/hoteles/{hotel_id}    | Admin    | Edita un hotel.                                                                   |
| DELETE | /api/hoteles/{hotel_id}    | Admin    | Elimina un hotel.                                                                 |
| GET    | /api/paquetes              | Sesión   | Paquetes cerrados.                                                                |
| POST   | /api/paquetes              | Admin    | Crea un paquete.                                                                  |
| PUT    | /api/paquetes/{paquete_id} | Admin    | Edita un paquete.                                                                 |
| DELETE | /api/paquetes/{paquete_id} | Admin    | Elimina un paquete.                                                               |
| GET    | /api/vuelos                | Personal | Programación de vuelos con plazas libres.                                         |
| POST   | /api/vuelos                | Admin    | Programa un vuelo.                                                                |
| GET    | /api/vuelos/{vuelo_id}     | Personal | Detalle de un vuelo.                                                              |
| PUT    | /api/vuelos/{vuelo_id}     | Admin    | Edita un vuelo.                                                                   |
| DELETE | /api/vuelos/{vuelo_id}     | Admin    | Elimina un vuelo sin reservas ni paquetes (si los tiene, se cancela o desactiva). |

## Reservas · app/routers/reservas.py

| Método | Ruta                                | Acceso   | Descripción                                                                                                                |
|--------|-------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------|
| GET    | /api/reservas                       | Personal | Todas las reservas, con búsqueda y filtros por estado y pago.                                                              |
| POST   | /api/reservas                       | Sesión   | Registra la reserva con su venta y su factura; el personal puede reservar a nombre de un cliente y cobrar en el mostrador. |
| POST   | /api/reservas/cotizar               | Sesión   | Precio exacto y plazas libres de un viaje, sin guardar nada.                                                               |
| GET    | /api/reservas/mias                  | Sesión   | Reservas de la cuenta de la sesión.                                                                                        |
| GET    | /api/reservas/{reserva_id}          | Sesión   | Detalle completo de una reserva (la propia; el personal, cualquiera).                                                      |
| PUT    | /api/reservas/{reserva_id}          | Personal | Edita una reserva: recalcula precios y plazas.                                                                             |
| DELETE | /api/reservas/{reserva_id}          | Sesión   | Elimina una reserva sin pagar; su venta y su factura quedan anuladas.                                                      |
| POST   | /api/reservas/{reserva_id}/cancelar | Sesión   | Cancela una reserva: el cliente, las no pagadas; el personal, también las pagadas.                                         |
| PATCH  | /api/reservas/{reserva_id}/estado   | Personal | Confirma o cancela una reserva.                                                                                            |

## Pasajeros · app/routers/pasajeros.py

| Método | Ruta                                 | Acceso | Descripción                                      |
|--------|--------------------------------------|--------|--------------------------------------------------|
| GET    | /api/reservas/{reserva_id}/pasajeros | Sesión | Datos de los pasajeros de una reserva.           |
| PUT    | /api/reservas/{reserva_id}/pasajeros | Sesión | Registra o reemplaza los datos de los pasajeros. |

| Método | Ruta                              | Acceso   | Descripción                                 |
|--------|-----------------------------------|----------|---------------------------------------------|
| GET    | /api/vuelos/{vuelo_id}/manifiesto | Personal | Manifiesto de pasajeros de un vuelo en CSV. |

## Pagos · app/routers/pagos.py

| Método | Ruta                                      | Acceso   | Descripción                                                                           |
|--------|-------------------------------------------|----------|---------------------------------------------------------------------------------------|
| POST   | /api/pagos/stripe/webhook                 | Pública  | Recibe los avisos de Stripe, verifica la firma y registra el pago.                    |
| POST   | /api/reservas/{reserva_id}/pago/checkout  | Sesión   | Crea o reutiliza la sesión de Stripe Checkout de la reserva (un solo enlace abierto). |
| POST   | /api/reservas/{reserva_id}/pago/confirmar | Sesión   | Confirma el pago al volver de Stripe (respaldo del webhook; idempotente).             |
| POST   | /api/reservas/{reserva_id}/pago/manual    | Personal | Registra un pago en efectivo, por transferencia o con datáfono.                       |

## Ventas, facturas, reportes, dashboard, PQR y chatbot · app/routers/comercial.py

| Método | Ruta                           | Acceso   | Descripción                                                                       |
|--------|--------------------------------|----------|-----------------------------------------------------------------------------------|
| POST   | /api/chatbot                   | Sesión   | Asistente conversacional con IA, con historial por conversación.                  |
| GET    | /api/estadisticas              | Personal | Indicadores y series del dashboard, con filtros.                                  |
| GET    | /api/facturas                  | Sesión   | Facturas con filtros por número, cliente y fechas (el cliente solo ve las suyas). |
| GET    | /api/facturas/{factura_id}/pdf | Sesión   | Factura en PDF.                                                                   |
| GET    | /api/pqr                       | Sesión   | PQR (el cliente solo ve las suyas).                                               |
| POST   | /api/pqr                       | Sesión   | Registra una petición, queja o reclamo.                                           |
| PATCH  | /api/pqr/{pqr_id}              | Personal | Cambia el estado de una PQR y registra la respuesta.                              |
| GET    | /api/reportes/ventas           | Personal | Reporte de ventas de un día o rango en JSON, PDF o Excel.                         |
| GET    | /api/ventas                    | Sesión   | Historial de ventas con filtros (el cliente solo ve las suyas).                   |
| POST   | /api/ventas                    | Personal | Venta de mostrador de productos y servicios.                                      |
| GET    | /api/ventas/{venta_id}         | Sesión   | Detalle de una venta.                                                             |

## Recomendaciones con IA · app/routers/recomendaciones.py

| Método | Ruta                          | Acceso | Descripción                                                        |
|--------|-------------------------------|--------|--------------------------------------------------------------------|
| POST   | /api/destinos/recomendaciones | Sesión | Tres destinos recomendados por IA según los intereses del viajero. |

## Contacto · app/routers/contacto.py

| Método | Ruta          | Acceso  | Descripción                         |
|--------|---------------|---------|-------------------------------------|
| GET    | /api/contacto | Admin   | Mensajes de contacto recibidos.     |
| POST   | /api/contacto | Pública | Mensaje del formulario de contacto. |

## Estado del servicio · app/main.py

| Método | Ruta        | Acceso  | Descripción                                                          |
|--------|-------------|---------|----------------------------------------------------------------------|
| GET    | /api/health | Pública | Estado de la API y de la base (usado por el healthcheck de Railway). |
| GET    | / · /salud  | Pública | Presentación y comprobación rápida.                                  |

# 7 Seguridad

La seguridad se aplica en el servidor: el frontend solo oculta lo que el usuario no puede usar.

## 7.1 Autenticación y control de acceso

- JWT HS256 con clave obligatoria, expiración, algoritmo fijo y versión de sesión (ver 6.2).
- Autorización por dependencias en cada ruta: `Administrador`, `Empleado0Admin` y `UsuarioActual`. Los recursos del cliente (reservas, facturas, ventas y PQR) se filtran por su cuenta en el servidor.
- El registro público ignora cualquier rol enviado y crea clientes; las pruebas lo verifican (`test_el_registro_no_asigna_rol_de_personal`).
- Debe existir siempre al menos un administrador activo; nadie puede cambiar su propio rol ni eliminar su propia cuenta.

**Auditoría de acceso (24 de septiembre de 2026).** De las 84 rutas, 12 son públicas y todas intencionales: login, registro, recuperar y restablecer contraseña, catálogo de solo lectura, formulario de contacto y webhook de Stripe (con firma). Comprobado en producción: un visitante sin sesión recibe 401 en las rutas de administración, y una cuenta de cliente recibe 403 en todas ellas, incluido el intento de asignarse el rol de administrador.

## 7.2 Contraseñas y recuperación

- Hash con **pwdlib (Argon2)**, el esquema recomendado; nunca se guardan ni se registran en claro.
- Política: de 8 a 128 caracteres, mayúsculas y minúsculas, al menos un número y un carácter especial, sin espacios en los bordes, no puede ser una clave común ni contener el correo o el nombre.
- Recuperación: la respuesta es idéntica exista o no la cuenta (no revela qué correos están registrados); el enlace dura una hora y sirve una sola vez.
- El arranque detecta si el administrador conserva la clave de ejemplo y lo obliga a cambiarla.

## 7.3 Límites de peticiones

Ventana deslizante en memoria (`core/limitador.py`), con la IP real detrás del proxy de Railway (`PROXIES_DE_CONFIANZA=1`). Superar un límite responde 429 con `Retry-After`.

| Acción                 | Límite                                                                          |
|------------------------|---------------------------------------------------------------------------------|
| Iniciar sesión         | 10 cada 5 min por IP · 5 cada 5 min por IP y correo · 20 cada 15 min por correo |
| Registrarse            | 5 por hora por IP                                                               |
| Recuperar contraseña   | 5 cada 15 min por IP · 3 por hora por destinatario                              |
| Restablecer contraseña | 10 cada 15 min por IP                                                           |
| Cambiar contraseña     | 10 cada 15 min por IP · 5 cada 15 min por usuario                               |
| Formulario de contacto | 5 por hora por IP                                                               |

| Acción         | Límite                                             |
|----------------|----------------------------------------------------|
| Cotizar        | 180 cada 10 min por IP                             |
| Crear reservas | 30 por hora por usuario                            |
| Chatbot        | 60 cada 10 min por IP · 20 cada 10 min por usuario |

## 7.4 Cabeceras, CORS y otras defensas

| Medida                   | Detalle                                                                                                                                                                             |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSP de la API            | default-src 'none'; frame-ancestors 'none'; base-uri 'none'; form-action 'none'.                                                                                                    |
| CSP del frontend         | Generada en cada compilación en dist/_headers: solo recursos propios, la API como único destino de fetch, mapas de Google en frame-src, frame-ancestors 'none' y object-src 'none'. |
| Otras cabeceras          | X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer, Permissions-Policy, Cache-Control: no-store en la API y HSTS en HTTPS.                        |
| CORS                     | Solo los orígenes de ORIGENES_PERMITIDOS (la URL del frontend).                                                                                                                     |
| Tamaño de las peticiones | Máximo 1 MB (413).                                                                                                                                                                  |
| Inyección SQL            | Consultas parametrizadas con SQLAlchemy; las búsquedas escapan los comodines de LIKE. Hay pruebas con cargas maliciosas.                                                            |
| Pagos                    | Webhook de Stripe con firma HMAC-SHA256 y margen de tiempo; importe, moneda y reserva se validan contra la base. El navegador no decide si se pagó.                                 |
| IA                       | El historial del chatbot sale de la base: el cliente no puede inyectar mensajes con otro rol ni escribir en conversaciones ajenas.                                                  |
| Secretos                 | Solo en variables de entorno; .env está en .gitignore. La imagen Docker corre sin privilegios de root (uid 10001).                                                                  |

## 8 Módulos funcionales

Cómo resuelve el sistema cada flujo de negocio.

### 8.1 Catálogo, cotización y precios

El catálogo público lista los destinos activos y, por destino, sus vuelos de ida y regreso, hoteles, excursiones y paquetes. La cotización (POST /api/reservas/cotizar) usa exactamente el mismo cálculo que la reserva, así el precio mostrado nunca difiere del cobrado:

```
# services/precios.py
Paquete cerrado total = precio_base * pasajeros # hotel y excursiones incluidos
A la carta vuelo = tarifa_de_vuelo * pasajeros
hotel = precio_noche * noches * habitaciones # una habitación por cada 2 pasajeros
excursiones = Σ precio * cantidad
total = vuelo + hotel + excursiones # redondeo a centavos (Decimal)
```

### 8.2 Reservas

- Estados: pendiente → confirmada (al pagar) o cancelada. Pago: pendiente, pagado o fallido.
- Disponibilidad: cada vuelo vende hasta su capacidad; la reserva bloquea las plazas dentro de la misma transacción.
- El cliente cancela o elimina sus reservas no pagadas; el personal además edita (se recalculan precios y plazas), confirma y cancela las pagadas (el reembolso es manual).
- Datos de pasajeros opcionales, que no bloquean la reserva; una persona no puede figurar en dos reservas activas del mismo vuelo. El personal descarga el manifiesto CSV de cada vuelo.

### 8.3 Pagos

- Stripe Checkout en pesos colombianos (el importe se envía en centavos). Hay un solo enlace abierto por reserva y se cierra si se cobra por otra vía.
- Confirmación: el **webhook** firmado (checkout.session.completed y checkout.session.async\_payment\_succeeded) es la vía principal; la vuelta del navegador a /reservas/pago-exitoso es el respaldo. registrar\_pago() es idempotente.
- Cobro en mostrador: efectivo, transferencia o datáfono, con comprobante y registro de quién cobró.
- Al quedar pagada: reserva confirmada, venta completada y correo de confirmación con la factura en PDF, una sola vez.

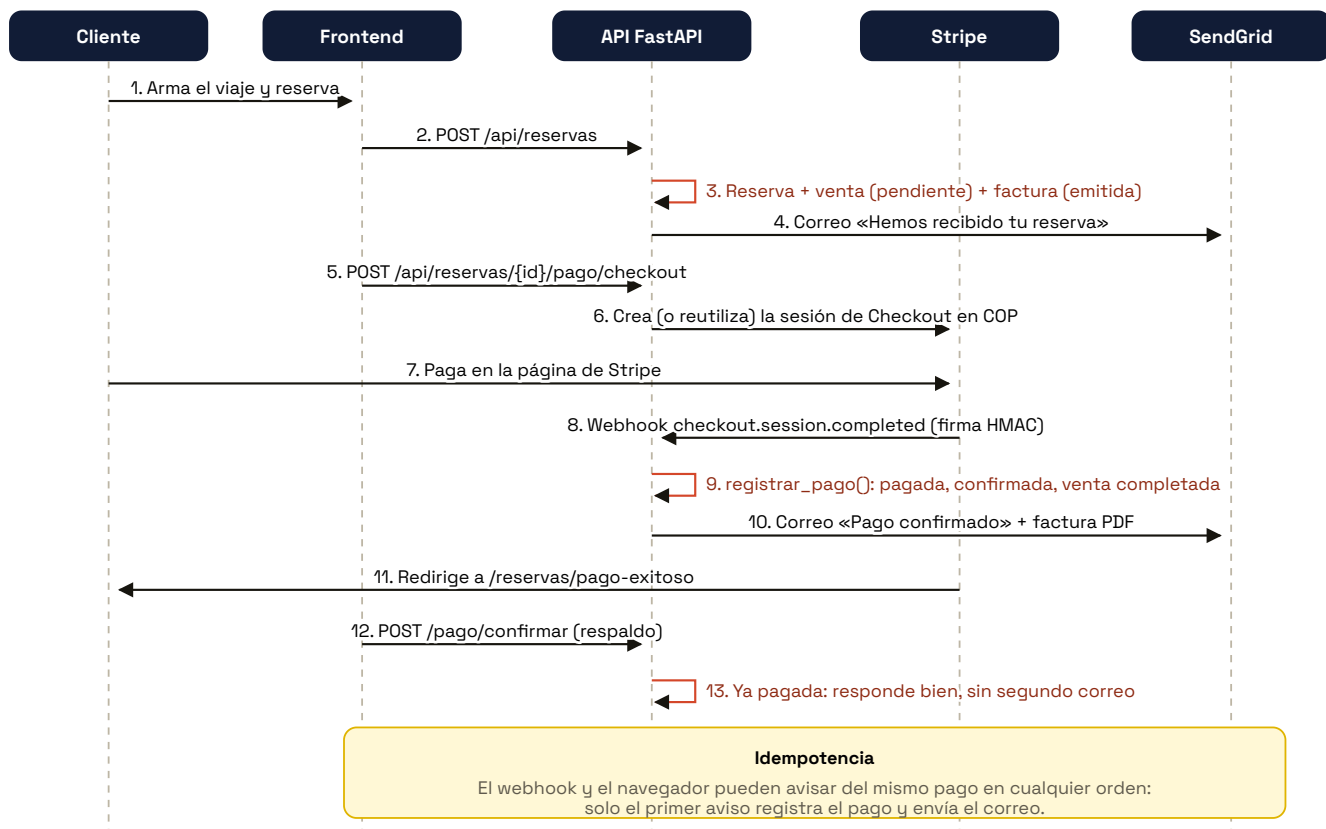


Figura 4. Pago con Stripe: el webhook confirma aunque el cliente cierre la pestaña.

## 8.4 Ventas, facturas y reportes

- Ventas de reservas (automáticas) y de mostrador (productos y servicios, con descuento e impuesto opcionales).
- Factura en PDF con emisor, cliente, líneas, subtotal, descuento, impuestos y total.
- Reporte de ventas por día o rango en JSON, PDF o Excel (hojas «Resumen» y «Ventas»). Las ventas canceladas se listan, pero no suman en los totales.

## 8.5 Dashboard y estadísticas

GET /api/estadisticas alimenta el panel del personal: clientes, reservas vendidas, facturación (con variación frente al periodo anterior), PQR pendientes, ticket promedio, pasajeros y catálogo activo; la serie de ventas (gráficos de barras y de línea) por día, semana o mes; la composición por concepto; las ventas por estado; y los destinos y clientes con más facturación. Filtros: fecha inicial y final, agrupación, estado, producto, servicio y cliente. Todo se calcula en el servidor; el frontend no contiene datos fijos.

## 8.6 PQR

Tipos: petición, queja y reclamo. Estados: pendiente, en proceso, respondida y cerrada. El cliente registra y consulta las suyas, con la respuesta de la agencia; el personal ve todas, cambia su estado y responde.

## 8.7 Chatbot y recomendaciones con IA

- La integración vive en el backend (`services/recomendaciones.py`): la clave nunca llega al navegador.
- Chatbot: guarda cada conversación en `conversaciones` y `mensajes`; envía como contexto los últimos 10 mensajes y admite hasta 200 por conversación. Tiene instrucciones para no inventar precios ni estados de reservas.
- Recomendaciones: el modelo elige 3 destinos del catálogo real y responde en JSON validado.
- Respaldo: sin clave o si el proveedor falla, el chatbot responde con un texto de ayuda y las recomendaciones usan coincidencia de palabras clave. La app nunca se cae por la IA.

## 8.8 Correos transaccionales

| Correo                    | Cuándo                                                                                                 |
|---------------------------|--------------------------------------------------------------------------------------------------------|
| Bienvenida                | Al registrarse.                                                                                        |
| Recuperar contraseña      | Al pedirlo, si la cuenta existe y está activa.                                                         |
| Hemos recibido tu reserva | Al registrar una reserva pendiente de pago.                                                            |
| Pago confirmado           | Al quedar pagada (Stripe, webhook o mostrador): todo el detalle del viaje y la factura en PDF adjunta. |

**Por qué por API y no por SMTP.** Railway bloquea el SMTP saliente en los planes que no son Pro. El backend envía por la API HTTPS de SendGrid; si solo existe la configuración SMTP de SendGrid (usuario `apikey`), reutiliza esa misma clave por HTTPS. Los correos salen en segundo plano y nunca bloquean la operación.

## 8.9 Paneles por rol

| Rol      | Barra lateral                                                                                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cliente  | Mi cuenta (Mis reservas, Mis facturas) · Atención (Mis PQR, Chatbot) · Acceso rápido (Ir al inicio, Reservar un viaje, Cambiar contraseña).                                       |
| Personal | Gestión (Reservas, Nueva reserva, Vuelos; y para el administrador Catálogo, Usuarios, Mensajes) · Comercial (Resumen, Reservas vendidas, Facturas, PQR, Chatbot) · Acceso rápido. |

# 9 Configuración

Toda la configuración entra por variables de entorno (core/configuracion.py). En local se leen de backend/.env, que nunca se sube al repositorio.

## Aplicación

| Variable             | Por defecto                                     | Descripción                                                                    |
|----------------------|-------------------------------------------------|--------------------------------------------------------------------------------|
| NOMBRE_APP           | Aurora Viajes API                               | Nombre de la API.                                                              |
| ENTORNO              | desarrollo                                      | desarrollo o produccion (la imagen Docker fija produccion).                    |
| DEPURACION           | false                                           | true publica /docs y registra cada consulta SQL. Siempre false en producción.  |
| ORIGENES_PERMITIDOS  | http://localhost:5173,<br>http://127.0.0.1:5173 | Orígenes admitidos por CORS (JSON o separados por comas): la URL del frontend. |
| FRONTEND_URL         | http://localhost:5173                           | URL pública del frontend: enlaces de los correos y retorno de Stripe.          |
| PROXIES_DE_CONFIANZA | 1                                               | Proxies delante de la app (Railway = 1) para leer la IP real en los límites.   |
| TAMANO_MAXIMO_CUERPO | 1000000                                         | Tamaño máximo del cuerpo de una petición, en bytes (413 si se supera).         |

## Base de datos

| Variable                            | Por defecto        | Descripción                                                                |
|-------------------------------------|--------------------|----------------------------------------------------------------------------|
| MOTOR_BD                            | mysql              | mysql o sqlite.                                                            |
| DATABASE_URL <span>secreta</span>   | —                  | URI completa de la base. En Railway: \${MySQL.MYSQL_URL}.                  |
| SQLITE_PATH                         | ./aurora_viajes.db | Archivo SQLite con MOTOR_BD=sqlite.                                        |
| MYSQL_HOST                          | 127.0.0.1          | Servidor MySQL (si no hay DATABASE_URL).                                   |
| MYSQL_PORT                          | 3306               | Puerto MySQL.                                                              |
| MYSQL_USER                          | root               | Usuario MySQL.                                                             |
| MYSQL_PASSWORD <span>secreta</span> | —                  | Contraseña MySQL (secreta).                                                |
| MYSQL_DATABASE                      | aurora_viajes      | Nombre de la base.                                                         |
| MYSQL_CHARSET                       | utf8mb4            | Juego de caracteres (utf8mb4).                                             |
| MYSQL_DRIVER                        | aiomysql           | Controlador asíncrono (aiomysql).                                          |
| BD_SIN_POOL                         | false              | Desactiva el pool de conexiones (entornos que cierran conexiones ociosas). |
| MYSQL_SSL                           | false              | Fuerza TLS hacia MySQL.                                                    |

| Variable                 | Por defecto | Descripción                                                            |
|--------------------------|-------------|------------------------------------------------------------------------|
| MYSQL_SSL_CA             | —           | Ruta del certificado de la CA.                                         |
| MYSQL_SSL_CA_PEM         | —           | Contenido PEM de la CA (para verificar el certificado del servidor).   |
| MYSQL_SSL_VERIFICAR_HOST | false       | Exige que el nombre del servidor coincida con el certificado.          |
| MIGRACION_AUTOMATICA     | true        | false pausa la migración de una base antigua (para respaldarla antes). |

## Seguridad y administrador

| Variable                            | Por defecto            | Descripción                                                                                        |
|-------------------------------------|------------------------|----------------------------------------------------------------------------------------------------|
| SECRET_KEY <span>secreta</span>     | —                      | Clave de firma de los JWT. Obligatoria, larga y aleatoria (secreta).                               |
| ALGORITMO_JWT                       | HS256                  | Algoritmo de firma de los JWT.                                                                     |
| MINUTOS_EXPIRACION_TOKEN            | 60                     | Duración de la sesión en minutos.                                                                  |
| ADMIN_EMAIL                         | admin@auroraviajes.com | Correo del administrador inicial; el arranque lo busca por este correo.                            |
| ADMIN_PASSWORD <span>secreta</span> | —                      | Clave del administrador al crearlo por primera vez (secreta; no usar la de ejemplo en producción). |
| ADMIN_RESTABLECER_CONTRASENA        | false                  | true: el siguiente arranque fija la clave del admin a ADMIN_PASSWORD (recuperar acceso).           |

## Pagos (Stripe)

| Variable                                   | Por defecto | Descripción                                    |
|--------------------------------------------|-------------|------------------------------------------------|
| STRIPE_SECRET_KEY <span>secreta</span>     | —           | Clave secreta de Stripe (sk_test_ o sk_live_). |
| STRIPE_WEBHOOK_SECRET <span>secreta</span> | —           | Clave de firma del webhook (whsec_).           |

## Correo

| Variable                              | Por defecto                           | Descripción                                                                       |
|---------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------|
| SENDGRID_API_KEY <span>secreta</span> | —                                     | Clave de la API de SendGrid. Con la configuración SMTP de SendGrid no hace falta. |
| SENDGRID_API_URL                      | https://api.sendgrid.com/v3/mail/send | Endpoint de envío de SendGrid.                                                    |
| SMTP_HOST                             | —                                     | Servidor SMTP (smtp.sendgrid.net).                                                |
| SMTP_PORT                             | 587                                   | Puerto SMTP.                                                                      |
| SMTP_USER                             | —                                     | Usuario SMTP (apikey en SendGrid).                                                |
| SMTP_PASSWORD <span>secreta</span>    | —                                     | Contraseña SMTP; en SendGrid es la clave de API (secreta).                        |

| Variable         | Por defecto              | Descripción                                        |
|------------------|--------------------------|----------------------------------------------------|
| SMTP_FROM        | noreply@auroraviajes.com | Remitente de los correos (verificado en SendGrid). |
| SMTP_FROM_NOMBRE | Aurora Viajes            | Nombre visible del remitente.                      |
| SMTP_USE_TLS     | true                     | STARTTLS en SMTP.                                  |

## Inteligencia artificial (Groq)

| Variable                                  | Por defecto                                     | Descripción                                                          |
|-------------------------------------------|-------------------------------------------------|----------------------------------------------------------------------|
| PROVEEDOR_IA_API_KEY <span>secreta</span> | —                                               | Clave del proveedor de IA (Groq). Sin ella se usa el respaldo local. |
| PROVEEDOR_IA_URL                          | https://api.groq.com/openai/v1/chat/completions | Endpoint compatible con OpenAI.                                      |
| PROVEEDOR_IA_MODELO                       | llama-3.1-8b-instant                            | Modelo de lenguaje.                                                  |
| PROVEEDOR_IA_TIMEOUT                      | 20.0                                            | Tiempo máximo de espera de la IA, en segundos.                       |
| PROVEEDOR_IA_REINTENTOS                   | 2                                               | Reintentos ante fallos pasajeros.                                    |

## Frontend

| Variable     | Por defecto | Descripción                                                                                                                   |
|--------------|-------------|-------------------------------------------------------------------------------------------------------------------------------|
| VITE_API_URL | /api        | URL de la API. Se incrusta al compilar y define el connect-src de la CSP. En producción: la URL de Railway terminada en /api. |

**En producción** define siempre SECRET\_KEY (larga y aleatoria), ADMIN\_PASSWORD propia (no la de ejemplo), DEPURACION=false, ORIGENES\_PERMITIDOS y FRONTEND\_URL con la URL real del frontend, y PROXIES\_DE\_CONFIANZA=1.

# 10 Instalación y ejecución local

Del repositorio a la aplicación funcionando en el equipo, sin tocar servicios reales.

## 10.1 Requisitos

- Git.
- Python 3.12 o superior.
- Node.js 22 LTS (Vite 8 requiere una versión reciente).
- Opcional: MySQL o MariaDB para probar contra MySQL.

## 10.2 Backend

```
git clone https://github.com/davidabb2000/aurora-viajes.git
cd aurora-viajes/backend
python -m venv .venv
.venv\Scripts\activate           # Windows · en Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
python -m scripts.desarrollo_local # API en http://127.0.0.1:8001 con SQLite, sin .env ni servicios
externos
```

`scripts.desarrollo_local` no lee `backend/.env`, desactiva correo, pagos e IA y usa `backend/aurora_dev.db`. Para trabajar con MySQL, copia `.env.example` a `.env`, ajusta la conexión y ejecuta `uvicorn app.main:app --reload`.

## 10.3 Frontend

```
cd aurora-viajes/frontend
npm install
npm run dev                # http://localhost:5173 · /api se redirige a 127.0.0.1:8001
(vite.config.js)
```

**Primer acceso local.** El administrador se crea con `ADMIN_EMAIL` y `ADMIN_PASSWORD`. Con la clave de ejemplo, la aplicación obliga a cambiarla en el primer ingreso.

# 11 Pruebas

300 casos automatizados (212 funciones de prueba en 16 archivos), una colección de Postman y análisis estático del frontend.

## 11.1 Pruebas automatizadas del backend

```
cd aurora-viajes/backend
python -m pytest # 300 passed
```

Son herméticas: tests/conftest.py fija las variables antes de importar la app, usa una SQLite temporal y aborta si detecta credenciales reales. Stripe, SendGrid y la IA se sustituyen por dobles de prueba.

| Archivo                | Cubre                                                                                          | Funciones |
|------------------------|------------------------------------------------------------------------------------------------|-----------|
| test_cuentas.py        | Registro, login, cambio y recuperación de contraseña, política de claves, cuentas del personal | 29        |
| test_catalogo.py       | Catálogo, destinos, vuelos, hoteles, excursiones, paquetes, productos y servicios              | 27        |
| test_seguridad.py      | Roles, inyección SQL, tokens manipulados, cabeceras, /docs deshabilitado                       | 23        |
| test_cotizacion.py     | Cálculo de precios y cotización                                                                | 14        |
| test_mostrador.py      | Reservas del personal y cobro en mostrador                                                     | 13        |
| test_arranque.py       | Arranque sin base, reintentos, migración pendiente                                             | 12        |
| test_pasajeros.py      | Datos de pasajeros y manifiesto                                                                | 12        |
| test_reservas.py       | Alta, edición, estados y cancelación de reservas                                               | 12        |
| test_configuracion.py  | Configuración y envío por SendGrid                                                             | 11        |
| test_correos.py        | Vía HTTPS, adjuntos, recuperación y correo de pago                                             | 11        |
| test_pagos.py          | Checkout, confirmación, webhook y correo de pago                                               | 11        |
| test_auth.py           | Tokens y sesión                                                                                | 9         |
| test_disponibilidad.py | Plazas por vuelo                                                                               | 8         |
| test_siembra.py        | Datos iniciales, idempotencia y administrador                                                  | 8         |
| test_chatbot.py        | Chatbot y aislamiento de conversaciones                                                        | 6         |
| test_comercial.py      | Ventas, reportes, facturas y estadísticas                                                      | 6         |

## 11.2 Colección de Postman

backend/postman/collections/aurora-quinto-avance.postman\_collection.json incluye 11 peticiones: login, registrar venta, historial filtrado, reporte PDF y Excel, consultar facturas, descargar factura, dashboard filtrado, registrar y gestionar PQR, y chatbot. Usa las variables baseUrl y el token obtenido en el login.

## 11.3 Frontend

```
cd aurora-viajes/frontend
npm run lint           # oxlint
npm run build          # compilación de producción y generación de dist/_headers
```

# 12 Despliegue

Backend y base en Railway, frontend en Cloudflare. El orden es base → backend → frontend, y al final se completan CORS y la URL del frontend en el backend.

## 12.1 Railway: base de datos y backend

1. Crear el proyecto y añadir el servicio **MySQL** (con volumen: los datos sobreviven a reinicios y despliegues).
2. Añadir el servicio del backend desde el repositorio de GitHub con *Root Directory* backend. Railway usa el Dockerfile y railway.json (healthcheck /api/health con 300 s de margen y reinicio ante fallos, hasta 10 veces).
3. Definir las variables: DATABASE\_URL=\${{MySQL.MYSQL\_URL}} (referencia a la red privada), MOTOR\_BD=mysql, SECRET\_KEY, ADMIN\_EMAIL, ADMIN\_PASSWORD, ORIGENES\_PERMITIDOS, FRONTEND\_URL, PROXIES\_DE\_CONFIANZA=1, DEPURACION=false y las de los servicios externos.
4. Cada *push* a main despliega de nuevo (unos 30 s). Al arrancar, el backend crea las tablas, migra si hace falta y siembra los datos iniciales.

## 12.2 Cloudflare: frontend

```
cd aurora-viajes/frontend
VITE_API_URL=https://aurora-backend-production-a2b4.up.railway.app/api npm run deploy
# compila, genera dist/_headers con la CSP y publica con Wrangler (sesión iniciada en la máquina)
```

**Importante.** Cloudflare no se reconstruye desde GitHub: cada cambio del frontend exige ejecutar `npm run deploy`.

## 12.3 Stripe

- STRIPE\_SECRET\_KEY: clave secreta (sk\_test\_... en pruebas, sk\_live\_... en real).
- Webhook: endpoint `https://aurora-backend-production-a2b4.up.railway.app/api/pagos/stripe/webhook` con los eventos `checkout.session.completed` y `checkout.session.async_payment_succeeded`; su clave de firma (whsec\_...) va en STRIPE\_WEBHOOK\_SECRET. No se debe confundir con la clave secreta de la API.
- Tarjeta de prueba: 4242 4242 4242 4242, cualquier fecha futura y cualquier CVC.

## 12.4 SendGrid

- Clave de API con permiso *Mail Send* (en SENDGRID\_API\_KEY o, con la configuración SMTP de SendGrid, en SMTP\_PASSWORD).
- Remitente verificado: *Settings* → *Sender Authentication* → *Single Sender*; esa dirección va en SMTP\_FROM.
- Con un remitente de correo gratuito (Gmail, Hotmail) los mensajes pueden llegar a Spam: la solución definitiva es un dominio propio con *Domain Authentication*.

## 12.5 Verificación después de desplegar

| Qué                    | Cómo                                                                                                               |
|------------------------|--------------------------------------------------------------------------------------------------------------------|
| API y base             | GET <code>https://aurora-backend-production-a2b4.up.railway.app/api/health</code> → baseDeDatos: lista.            |
| Despliegue del backend | Estado del commit en GitHub ( <code>/commits/&lt;sha&gt;/status</code> ) o el panel de Railway.                    |
| Logs                   | <code>railway logs -s aurora-backend -d</code> (cada petición lleva su identificador).                             |
| Frontend publicado     | Cabecera <code>content-security-policy</code> y el nombre del bundle <code>assets/index-*.js</code> de la portada. |
| Correo                 | Registrarse o pedir recuperación; en el log, «Correo transaccional enviado ... por SendGrid».                      |
| Webhook                | Un aviso sin firma válida debe responder 400 (con 503, falta <code>STRIPE_WEBHOOK_SECRET</code> ).                 |

# 13 Operación y solución de problemas

Síntomas frecuentes, su causa y cómo resolverlos.

| Síntoma                                             | Causa probable                                                                     | Solución                                                                                                                                                     |
|-----------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /api/health dice «sin conexion» y todo responde 503 | La base no responde.                                                               | Revisar que el servicio MySQL esté activo y que DATABAS E_URL sea la referencia correcta. El backend reintenta solo; el log indica el error del controlador. |
| «migracion pendiente»                               | MIGRACION_AUTOMATICA=false con una base antigua.                                   | Hacer el respaldo y volver a activar la migración.                                                                                                           |
| El backend reinicia en bucle                        | Falta SECRET_KEY.                                                                  | Definirla y redespargar.                                                                                                                                     |
| No llega el correo de recuperación                  | La cuenta no existe o está inactiva, el correo cayó en Spam o SendGrid lo rechazó. | Revisar el log: «enviado por SendGrid» o el motivo del rechazo (403 = remitente sin verificar).                                                              |
| Los pagos no se confirman solos                     | Webhook sin clave o con la clave equivocada.                                       | La clave debe empezar por whsec_; el log mostrará 400 «firma no válida» si no coincide.                                                                      |
| Los cambios del frontend no se ven                  | Cloudflare no despliega desde GitHub.                                              | Ejecutar npm run deploy en frontend/.                                                                                                                        |
| Error 429                                           | Límite de peticiones.                                                              | Esperar el tiempo de Retry-After.                                                                                                                            |
| Se perdió el acceso de administrador                | Contraseña olvidada.                                                               | ADMIN_RESTABLECER_CONTRASENA=true, redespargar, entrar con ADMIN_PASSWORD, cambiarla y volver a false.                                                       |
| Cambiar el correo del administrador                 | —                                                                                  | Editarlo en Usuarios y actualizar ADMIN_EMAIL; si no se actualiza, el arranque avisa en el log y no crea otro administrador.                                 |

## 13.1 Mantenimiento

- **Respaldos:** la base no está expuesta a internet. Para un volcado, abrir un proxy TCP temporal (railway tcp-proxy create), ejecutar mysqldump y cerrarlo.
- **Costos:** Railway cobra por consumo y el servicio MySQL corre las 24 horas. Conviene fijar un límite de gasto y revisar el uso.
- **Claves:** si una clave se expone, rotarla en el proveedor y en Railway; cambiar SECRET\_KEY cierra todas las sesiones.
- **Dependencias:** actualizar con las pruebas en verde (pytest, npm run lint y npm run build) antes de desplegar.

## 14 Glosario

| Término      | Definición                                                                                       |
|--------------|--------------------------------------------------------------------------------------------------|
| 3FN          | Tercera forma normal: cada dato se guarda una sola vez y depende solo de la llave de su tabla.   |
| API REST     | Interfaz HTTP basada en recursos y métodos (GET, POST, PUT, PATCH, DELETE) que intercambia JSON. |
| CORS         | Mecanismo del navegador que limita qué orígenes pueden llamar a una API.                         |
| CSP          | Content Security Policy: cabecera que restringe de dónde puede cargar recursos una página.       |
| Healthcheck  | Comprobación periódica de que el servicio responde (/api/health).                                |
| Idempotencia | Repetir una operación produce el mismo resultado que hacerla una vez.                            |
| JWT          | JSON Web Token: credencial firmada que identifica la sesión.                                     |
| ORM          | Mapeo objeto-relacional: clases de Python que representan tablas (SQLAlchemy).                   |
| PQR          | Peticiones, quejas y reclamos.                                                                   |
| SPA          | Aplicación de una sola página: el navegador carga la app una vez y cambia de vista sin recargar. |
| Webhook      | Llamada HTTP que un servicio externo (Stripe) hace a la API para avisar de un evento.            |